Product & Interview Report: Global Freight Network Optimization & SLA Penalty Mitigation

This document serves two audiences. ***Part 1*** *explains the project in plain business language for recruiters and non-technical stakeholders.* ***Part 2*** *is a detailed technical defense of every major tool choice, written for interview preparation — including the specific engineering problems that came up and exactly how they were solved.*

Table of Contents

PART 1: The Business Product Report (Layman's Terms)

The Corporate Problem

Most supply chains are still run on **static rules**: "when stock falls below X units, order Y more." This sounds reasonable, but it quietly bleeds money in three specific ways.

- A static rule is set once, from past averages. It has no way of noticing a demand spike is coming — it only reacts after the shelf is already empty.
- A static rule has no concept of **capacity**. It might tell a factory to "order more," with no awareness that the factory can only physically produce a fixed number of units per day. Demand doesn't care about that limit, but a static rule doesn't either.
- A stockout is not just a missed sale. In most B2B supply contracts, failing to deliver on time triggers a **Service Level Agreement (SLA) penalty** — a real, contractually-owed cash payment, separate from and on top of the lost revenue.

Put together: when demand outpaces what a factory can produce, and nobody saw it coming, the business doesn't just lose a sale — it writes a penalty check. This project exists to replace the guessing with a system that sees the spike coming and calculates, in dollars, the cheapest way to avoid the penalty.

The Data Engine (Dataset)

Traditional planning tools work off static spreadsheets — a snapshot of last month's or last quarter's sales, manually exported and reviewed periodically. By the time anyone looks at it, the data is already old.

This project replaces that with a **live, continuously-running simulator** that behaves like a real chain of cash registers:

- It generates a new, realistic sale every fraction of a second — a specific product, a quantity, a store location, and an exact timestamp.
- It uses a data-generation library called Faker to produce realistic (not repetitive or robotic-looking) store locations and transaction identifiers.
- Because it never stops, every other part of the system always has fresh data to react to — there is no "waiting for the weekly export."

This is the foundation the rest of the system depends on: a business can only plan ahead of a demand spike if it can actually see the spike happening in near real time, not three weeks later in a spreadsheet.

The Implementation (End-to-End)

Here is what happens, in order, from the moment one simulated sale occurs to the moment the dashboard reflects it:

- A simulated cash register generates a sale and publishes it as a small message — think of it as a digital receipt.
- That message travels through a fast, reliable message delivery system (Redpanda) that sits between every part of the pipeline, like a postal service for programs. No program ever talks directly to another; everything passes through here.
- A forecasting service is always listening on that channel. It continuously builds up a running history of recent sales for each product, and periodically — once enough new sales have arrived — it predicts how much of each product will be needed over the next 7 days.
- That 7-day prediction is itself sent as a message through the same delivery system to an optimization service.
- The optimization service runs a mathematical model that decides the single cheapest possible production plan for the next 7 days — how much to manufacture each day — while respecting the factory's daily production limit and the warehouse's storage limit.
- That plan is saved into a small, fast local database.
- A live dashboard checks that database every 3 seconds and displays the current plan: how much to produce each day, how much inventory that leaves on hand, the total cost, and — most importantly — a clear red warning if any customer demand is going to go unmet.

No person touches any step of this. A single sale, generated automatically, can flow all the way through forecasting and optimization and appear as an updated plan on the dashboard within seconds.

The Outcome & Business Insights

The most valuable capability this system demonstrates is that it **looks ahead and prepares**, rather than only reacting to today.

Here is a concrete, real example pulled directly from this system's own test results. Imagine a factory that can make at most 150 units a day. Demand is a trickle on day one — just 10 units — but spikes to 300 units on day two.

- A traditional, reactive system only ever looks at today. On day one it makes 10 units (exactly what's needed). On day two, it can only make 150 more units, no matter how high demand is. That leaves 150 units of customer demand completely unmet, triggering $15,000 in SLA penalties on top of production costs — **$16,600 total**.
- This system looks at both days simultaneously. It recognizes that making the full 150 units on day one — even though day one only needs 10 — lets it bank the extra 140 units in the warehouse overnight, ready for day two.
- Storing those 140 extra units for one night costs a small holding fee: $1.50 per unit, or $210 total. Combined with day two's own full production run, this shrinks the shortfall from 150 units down to just 10 units — the true mathematical minimum possible, given the factory's total two-day capacity.
- Total cost: **$4,210** — a **74.6% reduction** — achieved automatically, with no human predicting the spike or manually deciding to build extra stock.

The business math in one sentence: the SLA penalty is $100 per unit, and holding a unit in the warehouse costs $1.50 per unit per day — meaning it would take **more than 66 days** of storage to cost as much as a single stockout penalty. Once you see the numbers laid out this way, "hold a little extra stock ahead of a known spike" stops being a judgment call and becomes an obvious, provable financial decision. This system makes that calculation automatically, every single time new demand data arrives, for every product, without anyone having to notice the spike coming themselves.

PART 2: Interview Preparation & Tech Stack Defense

Redpanda (via kafka-python)

What it is: Redpanda is a modern streaming data platform that speaks the same protocol as Apache Kafka, the industry-standard "message bus" technology. Because it's protocol-compatible, standard Kafka client libraries — including Python's `kafka-python`, used throughout this project — work against it with no code changes.

Its role in this project: Redpanda is the central nervous system. Every one of the four services communicates exclusively through two Redpanda topics, `live_orders` and `demand_forecasts` — never directly with each other. This means any single service can be stopped, restarted, or crash outright, and the rest of the pipeline keeps running undisturbed, catching back up from durable, persisted messages once it returns.

The Interview Defense

- Traditional Apache Kafka runs on the Java Virtual Machine and requires a separate coordination service (historically ZooKeeper, more recently a KRaft quorum) — meaning even a "minimal" Kafka setup is really two or three coordinated Java processes, each with its own memory footprint and garbage-collection pauses.
- Redpanda reimplements the Kafka wire protocol from scratch in C++ on the Seastar framework — the same thread-per-core, no-garbage-collector design used by high-performance databases like ScyllaDB. The result is a single binary with no JVM and no separate coordination service, even at full production scale.
- Redpanda ships genuine multi-architecture container images, so on Apple Silicon this project's broker runs as a native ARM64 binary — not translated through Rosetta or QEMU emulation, which is a well-known, measurable source of sluggish local Kafka setups on M-series hardware.
- **The one-line defense:** "I chose Redpanda because it's a drop-in replacement for Kafka at the protocol level — the exact same client code — while removing the JVM and cluster-coordination overhead that makes vanilla Kafka painful to run locally, and it runs natively on ARM64 rather than under emulation."

Prophet

What it is: Prophet is an open-source time-series forecasting library originally built at Meta, designed specifically for business data with trends and recurring seasonal patterns — the kind of data retail sales naturally produce.

Its role in this project: `demand_forecaster.py` fits one Prophet model per product against its recent rolling sales history, and asks it to forecast demand for the next 7 days.

The Interview Defense

This is worth understanding precisely, because the real story is more interesting — and more defensible in an interview — than "Prophet had a bug." **It didn't.** Prophet did exactly what it was configured to do; the mistake was a mismatch between the data's time scale and the forecast horizon, and catching that mismatch is the actual engineering win here.

- Prophet's default growth mode fits a straight-line trend through the historical data and extrapolates that same slope forward into the future.
- To make this project demoable without waiting real days for real data, order volume is bucketed into minute-level time buckets — meaning the "trend" Prophet was fitting was really a minute-to-minute rate of change.
- Extrapolating a minute-to-minute slope across a 7-day forecast horizon is a roughly 1,440-times larger time gap than the data it was fit on. This was tested directly: a data series of just four points, with values between 10 and 25 units, produced a day-7 forecast of **over 30,000 units** under Prophet's default linear growth — confirmed by rerunning the exact test again while writing this document.
- The fix was switching the model to `growth="flat"`, which forecasts around the recently observed average level instead of extrapolating a trend line. With only a handful of data points, there isn't enough statistical evidence to trust a directional trend in the first place — flat growth is the honest choice until enough real multi-day history accumulates, at which point `growth="linear"` (or `"logistic"`) becomes the right call again.
- **The one-line defense:** "This wasn't a library defect — it ran without a single error, which is exactly what made it dangerous. I caught it by sanity-checking the actual forecasted numbers against the input data, not by trusting that 'no exception' meant 'correct.' Fixing it meant understanding Prophet's growth model well enough to pick the right one for the amount of data actually available, not just accepting the default."

Google OR-Tools (GLOP)

What it is: OR-Tools is Google's open-source Operations Research toolkit. GLOP is specifically its linear programming (LP) solver — software that finds the single, mathematically provable *best* answer to a problem with costs and hard constraints, not a predicted or estimated one.

Its role in this project: Every time a fresh 7-day forecast arrives, `inventory_optimizer.py` uses GLOP to solve, from scratch, the exact daily production plan that minimizes total cost, subject to a hard daily factory output limit and a hard warehouse storage limit.

The Interview Defense

- **Operations Research vs. Machine Learning, precisely stated:** Machine Learning (Prophet, in this project) is *predictive* — it answers "what is likely to happen." Operations Research (GLOP, here) is *prescriptive* — it answers "given what's likely to happen, and given our real-world constraints and costs, what is the best decision we can make." This project deliberately chains both, in that order: predict, then decide.
- LP was chosen specifically because this is a constrained cost-minimization problem with a mathematically provable optimum — an LP solver finds that exact optimum in milliseconds. No amount of machine learning guarantees an *optimal* answer here, because ML recognizes patterns; it doesn't verify optimality against explicit constraints and costs.
- **The Slack Variable, in detail:** the original version of this model let the factory produce an unlimited number of units per day, so the model could always manufacture its way out of any demand spike — it was never actually capable of failing. Introducing a real 150-unit daily production cap changed that: on a day where demand exceeded what 150 units plus existing inventory could cover, the model had no way to balance its equations without inventory going negative — which is not allowed. Mathematically, the solver would return `INFEASIBLE`: no solution exists, at exactly the moment a plan was most needed.
- The fix was introducing a new decision variable, `S_t` — unmet demand, bounded at zero and above with no upper limit — and rewriting the core inventory balance equation so `S_t` can absorb any shortfall the factory truly cannot cover. Critically, `S_t` was also added into the cost function at $100 per unit, so the solver never uses it "for free" — it only ever produces a stockout once daily production is already maxed out, because one more unit of production ($10) is always cheaper than one unit of penalty ($100).
- The result: the model can never again return "no answer." It always finds a solution, and that solution always states, in exact dollars, what a capacity shortfall is costing the business.
- **The one-line defense:** "The slack variable is a classic Operations Research pattern for turning a hard infeasibility into a soft, priced business cost. I didn't just report a stockout number after the fact — I made it a real decision variable inside the objective function, so the solver actively weighs stockout cost against production cost against holding cost as three genuine trade-offs, and always picks the cheapest combination."

DuckDB

What it is: DuckDB is an embedded, in-process analytical database — often described as "SQLite for analytics." It runs directly inside the application process; there is no separate database server to install, configure, or connect to over a network.

Its role in this project: `inventory_optimizer.py` writes every newly-solved 7-day schedule into a local DuckDB file. `dashboard.py` reads from that same file to render the live Control Tower.

The Interview Defense

This is the single best engineering story in the project, because it's a bug that would never show up in a code review or a unit test — only by actually testing real concurrent behavior.

- DuckDB was chosen over a client-server database like Postgres because it needs zero setup — a single file, no server process to run or manage — while still being genuinely fast at the aggregation-heavy queries a dashboard needs, fast enough to query fresh on every single poll rather than requiring a caching layer in front of it.
- The "obvious" design for a Streamlit dashboard is to open one database connection when the app starts and reuse it for every refresh — this is the standard pattern in almost every framework, including Streamlit's own recommended connection-caching approach.
- That design was tested directly, using two separate, real operating-system processes: one holding a read-only connection open and polling it once a second, the other writing and committing a change partway through that polling loop. The long-held read-only connection never saw the update — not once, across ten separate polls over ten seconds.
- The explanation is MVCC (Multi-Version Concurrency Control), the concurrency model DuckDB's storage engine is built on: a connection captures a consistent snapshot of the database at the point it's opened, and — as directly observed here — a read-only connection's snapshot does not automatically advance to include commits made by other processes afterward. A brand new connection is required to see a new snapshot.
- Why this mattered: left uncaught, the dashboard would have looked completely healthy from the outside — the auto-refresh timer firing exactly every 3 seconds, the page visibly redrawing — while silently displaying the exact same stale numbers forever, with no error and no warning. That is a uniquely dangerous class of bug: one indistinguishable from correct behavior unless you know to check the actual values.
- The fix: the dashboard's data-loading function opens a brand new read-only connection on every single poll, runs its query, and closes it immediately. This costs a few extra milliseconds of connection overhead every 3 seconds — a negligible price for guaranteeing the dashboard is never silently wrong.
- **The one-line defense:** "This is the finding I'm proudest of in the whole project, because it only surfaces when you test two real concurrent processes against each other rather than reasoning about it in the abstract. I don't trust a concurrency assumption until I've watched it fail — or hold — under an actual concurrent test."

Streamlit & Plotly

What it is: Streamlit is a Python framework that turns a plain script into an interactive web application with no HTML, CSS, or JavaScript required. Plotly is a charting library that produces interactive charts — hover tooltips, zoom, pan — rather than static images.

Its role in this project: `dashboard.py` is the Control Tower: the single screen a business stakeholder would actually look at, showing live cost and risk KPIs, an interactive production-versus-stockout chart, and the raw schedule data.

The Interview Defense

- Speed of delivery was the deciding factor: a full B2B-style operational dashboard — KPI tiles, an interactive chart, a data table, live auto-refresh — was built in a single Python file, with no separate frontend build step, no JavaScript framework, and no API layer between frontend and backend. For a fast-moving internal tool or a portfolio project, the return on investment for a hand-built React frontend simply isn't there.
- **On auto-refresh, precisely:** Streamlit's frontend and backend already communicate over a WebSocket connection by default — that part isn't something this project added. What the `streamlit-autorefresh` package adds is a client-side timer that automatically triggers a script rerun every 3 seconds, without a full page reload, which is what preserves things like the selected product in the dropdown across refreshes.
- This is an important, honest distinction worth stating precisely in an interview: what's implemented here is **polling, not push**. The dashboard asks "anything new?" every 3 seconds; it is not proactively notified the instant `inventory_optimizer.py` writes a fresh schedule. A genuinely real-time push architecture would need the backend to notify connected clients directly, typically through a publish/subscribe layer. At a 3-second interval the difference is invisible to a human user, but polling is dramatically simpler to build and reason about, and was the right, deliberate tradeoff for this system.
- Plotly, specifically, was chosen over Streamlit's built-in chart types for the one visualization that needed real customization: a stacked bar (production plus stockout) combined with an overlaid inventory line and a dashed reference line marking the factory's exact capacity ceiling, all sharing one unified hover tooltip. That level of compositing isn't available from Streamlit's native charting.
- **The one-line defense:** "I chose Streamlit and Plotly so I could spend engineering time on the parts of this project that actually needed it — the streaming pipeline, the forecasting logic, the optimization model — instead of hand-building UI plumbing. And I can speak precisely to what 'live' means here: it's fast polling, not push, and that was a deliberate, defensible tradeoff, not an oversight."